STRIDEMAP.AI | AI Agent Governance Infrastructure

AgentGuard

Product Roadmap v1.0

MVP + Option A (Governance Layer) + Option C (Runtime Security)

Confidential | stridemap.ai | hello@stridemap.ai

> *"Think Palo Alto Prisma, but for AI agents — with a clear path to full runtime security."*

Document	AgentGuard Product Roadmap v1.0
Status	Active — Sprint 1 ready to begin
Owner	Stridemap (stridemap.ai)
Scope	MVP + Option A (AI Agent Governance) + Option C (Agent Runtime Security)
Founder background	13 yrs Cisco SDWAN, Citrix, Palo Alto NGFW + Agentic AI bootcamp
Target market	Bay Area enterprises — Financial services, Healthcare, Legal, Enterprise IT
Total timeline	~24 sprints across 6 months, full-time weekdays, weekends protected

NAVIGATION

Table of Contents

1	Executive Summary	3
2	Product Vision & Competitive Position	4
3	Full Tech Stack	5
4	Monorepo & Test Framework Architecture	6
5	Sprint 1 — Foundation & Landing Page	7
6	MVP — Core Governance (Sprints 2–4)	8
7	Option A · Phase 1 — Governance Layer (Sprints 5–8)	10
8	Option A · Phase 2 — Pilot Ready (Sprints 9–12)	12
9	Option A · Phase 3 — Production Scale (Sprints 13–16)	14
10	Option C — Agent Runtime Security (Sprints 17–24)	16
11	Go-to-Market & Job Search Strategy	18
12	Daily Routine & Development Workflow	19
13	Cost & Resource Plan	20
14	Risk Register	21
15	Success Metrics & Milestones	22

SECTION 1

Executive Summary

Stridemap builds security infrastructure for the age of agentic AI. This roadmap covers three sequential phases — MVP, Option A, and Option C — each building on the foundations of the previous one.

MVP	Sprints 1–4 (~4 wks)	Repo + landing page on stridemap.ai, LangGraph interceptor, policy engine, DLP, prompt injection detection, tamper-proof audit log, Streamlit dashboard. Demo-ready, OSS launch.
Option A	Sprints 5–16 (~12 wks)	AI Agent Governance Layer — production-grade: FastAPI, Postgres, RBAC, Docker, Kubernetes, SIEM integration, SSO, SOC2/HIPAA templates, PyPI release, policy marketplace, Next.js dashboard.
Option C	Sprints 17–24 (~8 wks)	Agent Runtime Security — OS-level sandboxing, tool call isolation, network egress control at kernel level, process spawn prevention. The evolution from governance to true runtime security.

Sequencing logic: Option A (governance layer) is built first because it directly leverages 13 years of Cisco SDWAN, Citrix, and Palo Alto NGFW expertise. It solves an immediate enterprise pain point and creates the interceptor, policy engine, and audit chain that Option C (runtime security) then extends into OS-level sandboxing. Option C is the natural technical evolution — not a separate product.

Solopreneur constraints respected: Full-time weekdays only, weekends protected. Total cash cost across 6 months: ~$200. Uses free/OSS stack throughout — Ollama for local LLM, Streamlit for MVP dashboard, GitHub Actions for CI, Vercel for landing page hosting.

SECTION 2

Product Vision & Competitive Position

The core insight: threat models for AI agents are nearly identical to network security problems solved over the past 20 years — data exfiltration, unauthorized access, lateral movement, policy enforcement. The attack surface has moved from packets to prompts. Stridemap applies proven NGFW thinking to this new layer.

Why Option A before Option C

Option A (AI Agent Governance) starts with a concrete enterprise problem that existing security teams understand immediately — visibility and policy enforcement. Option C (Runtime Security) is the long-term vision but requires Option A's interceptor architecture as its foundation. Building in sequence means each phase creates real, usable product — not just infrastructure for the next phase.

Competitive landscape

Competitor	What they do	AgentGuard gap / differentiator
HiddenLayer	ML model security	Not agent runtime — no tool-call-layer governance
Protect AI	AI supply chain security	No runtime DLP or policy enforcement for agents
CalypsoAI	LLM firewall, enterprise-only, closed	OSS, LangGraph-native, developer-first install
LangSmith	Observability only — no enforcement	Policy engine + DLP + injection + audit in one tool
Palo Alto Cortex	AI security — not OSS, not agent-native	Built by NGFW practitioner, open source, LangGraph-native
Wiz / CrowdStrike	Cloud/endpoint security pivoting to AI	Purpose-built for agents from day one, not retrofitted

AgentGuard's unique position: Built by an engineer with 13 years of enterprise network security testing (Cisco SDWAN, Palo Alto NGFW) who completed an Agentic AI bootcamp — the only person in the market who deeply understands both the NGFW policy enforcement model AND how LangGraph multi-agent orchestration works internally. This is not a software engineer learning security; it is a security engineer who learned AI agents.

SECTION 3

Full Tech Stack

Every technology choice prioritises zero-cost to start, modularity, and a clear upgrade path. SQLite in MVP becomes Postgres in v1. Streamlit dashboard becomes Next.js in v2. Local Ollama becomes OpenAI/Anthropic for production. Switching any layer is a config change, not a rewrite.

Agent Runtime	`LangGraph, LangChain`	Multi-agent orchestration — AgentGuard wraps this	MVP
Core Library	`Python 3.11, Presidio, spaCy, PyYAML, Pydantic v2, Loguru`	pip-installable AgentGuard package	MVP
LLM Providers	`Ollama (dev/free), OpenAI GPT-4o-mini (demo), Anthropic Claude`	Pluggable via LLMClient abstraction	MVP
Storage MVP	`SQLite + hashlib stdlib`	Zero-setup event store + SHA-256 audit chain	MVP
Storage v1+	`PostgreSQL 15, Alembic, Redis`	Production event store, migrations, async queue	v1
Dashboard MVP	`Streamlit, fpdf2`	Live trace viewer + PDF audit export	MVP
Dashboard v2	`Next.js 14, Tailwind, shadcn/ui, Recharts`	Full enterprise web app dashboard	v2
Landing Page	`Next.js 14, Tailwind CSS, Vercel`	stridemap.ai — deployed Sprint 1	Sprint 1
API Layer	`FastAPI, Uvicorn, JWT / python-jose`	REST API connecting dashboard to core library	v1
Testing	`pytest, pytest-asyncio, pytest-cov, Locust`	5-layer test framework — unit through adversarial	MVP
Code Quality	`Ruff, Black, pre-commit hooks`	Zero-lint policy enforced from day one	MVP
CI/CD	`GitHub Actions, Docker + Compose, Helm/K8s`	Web deploy + Python CI, staging → production	MVP→v2
Cloud v2+	`AWS ECS / GCP GKE, S3/R2, Render/Railway`	Enterprise VPC deploy, audit log archive	v2
Option C additions	`seccomp/ptrace, eBPF, gVisor (research)`	OS-level sandboxing — agent runtime isolation	Option C

M1 Pro advantage: Ollama runs llama3.2 at 30+ tokens/sec locally on the M1 Pro — free, private, zero API cost during all development and testing. Only switch to cloud LLM providers for final demo polish and production.

SECTION 4

Monorepo & Test Framework Architecture

Everything lives in a single GitHub repo: **github.com/stridemap/agentguard**. Landing page, Python core library, dashboard, API, demos, tests, and CI/CD — one commit history, one professional surface area. When a hiring manager or enterprise prospect clicks the link, they see a complete product.

Repo structure

Directory	Purpose
web/	Next.js landing page — stridemap.ai (Sprint 1)
agentguard/	pip-installable Python core library (Sprint 2+)
dashboard/	Streamlit MVP dashboard (Sprint 2+)
api/	FastAPI backend — REST API connecting dashboard to core (Sprint 6+)
demo/	Runnable named attack scenarios — finance, healthcare, legal, IT
policies/	Sample YAML policy files — default, finance, healthcare
tests/	5-layer test framework with plans, fixtures, evaluation
docs/	Architecture, security.md, setup.md, API reference
.github/workflows/	web-deploy.yml, python-ci.yml, nightly.yml, weekly-perf.yml

5-layer test framework

The test architecture directly applies 13 years of QA/performance testing expertise to the AI domain. Most AI builders have zero test coverage — this test suite is a visible differentiator in both enterprise sales and job interviews.

Layer	Cadence	What it covers	Tool	Runtime
1 — Unit	Every push	Each module in isolation — no API calls, mock LLM	pytest	<30 sec
2 — Integration	Daily	Modules working together — real SQLite, mock LLM	pytest	<2 min
3 — Scenario	Pre-demo + nightly	Named attack scenarios: PII_EXFIL, INJECT, UNAUTH	pytest	<5 min
4 — Performance	Weekly	Latency p99 <50ms, throughput >100 events/sec	Locust	<15 min

5 — Adversarial	Nightly	Crash recovery, malformed payloads, audit tamper	pytest	<5 min
+ Evaluation	Per release	Injection precision/recall, DLP accuracy, OWASP coverage	Custom	<10 min

Evaluation quality targets

Metric	Target	Gate?
Injection precision	> 0.90	Yes — blocks release
Injection recall	> 0.85	Yes — blocks release
DLP false positive rate	< 2%	Yes — blocks release
DLP false negative rate	< 1%	Yes — blocks release
Test coverage (core)	> 80%	Yes — blocks merge
Latency p99 overhead	< 50ms	Yes — documented SLA
Throughput sustained	> 100/sec	Advisory
OWASP LLM01–05 coverage	> 80%	Advisory

OWASP LLM Top 10 coverage map is maintained in `tests/plans/ATTACK_MATRIX.md` — every scenario test mapped to a specific OWASP LLM category. MVP targets >80% coverage of LLM01–LLM06.

SECTION 5

Sprint 1 — Foundation & Landing Page

Item	Detail
Goal	GitHub repo live, stridemap.ai on Vercel, CI green, full scaffold done
Duration	5 working days — Mon through Fri, 9am–5pm
Cost	~$0 (Vercel free, GitHub free, Zoho Mail free tier)
Deliverable	stridemap.ai publicly live — your first signal to the market

Daily plan

Mon Day 1	Create GitHub monorepo (agentguard). Full folder scaffold — web/, agentguard/, dashboard/, demo/, policies/, tests/, docs/. Copy Next.js landing page into web/. Root .gitignore + .env.example. First commit and push.	**GitHub repo live on github.com/stridemap/agentguard**
Tue Day 2	Vercel project created. Connect to GitHub repo, root directory = web/. Add environment variables. Deploy preview URL. Test all sections on mobile and desktop. Fix any render issues.	**Preview deploy URL live and shareable**
Wed Day 3	Connect stridemap.ai domain — add A and CNAME records at registrar. Verify HTTPS (Vercel auto-provisions SSL cert). Set up Zoho Mail free tier — hello@stridemap.ai and security@stridemap.ai. Update all mailto links in codebase.	**stridemap.ai live with HTTPS + professional email**
Thu Day 4	GitHub Actions CI pipeline for web/ — scoped with paths filter so only web/ changes trigger it. Runs: npm ci, ESLint, TypeScript check, next build. Vercel auto-deploy on main merge. Create GitHub issue templates (bug, feature, security). Write root README.	**CI/CD pipeline live — every push auto-deploys**
Fri Day 5	Scaffold all Python folders with stub README files: agentguard/, dashboard/, demo/, tests/ (with conftest.py, pytest.ini, TEST_PLAN.md, ATTACK_MATRIX.md stubs), docs/. Update LinkedIn headline to reference stridemap.ai and AgentGuard. Post: 'stridemap.ai is live — building AI agent governance.'	**Full repo scaffolded + first public announcement**

Sprint 1 tickets (create as GitHub issues)

- SETUP-001: Create GitHub monorepo and full folder scaffold
- WEB-001: Add Next.js landing page to web/ directory — verify install and dev
- WEB-002: Deploy web/ to Vercel and connect stridemap.ai custom domain
- WEB-003: Set up professional email at stridemap.ai via Zoho Mail free tier

- CI-001: GitHub Actions web CI pipeline — scoped to web/ path changes only
- SETUP-002: Scaffold Python package structure with stub README files
- SETUP-003: Write root README.md and update LinkedIn + GitHub profile bio

SECTION 6

MVP — Core Governance (Sprints 2–4)

Sprints	2, 3, 4 — 3 weeks after Sprint 1
Goal	Demo-ready AgentGuard with all 4 core modules + public OSS launch
Cost	~$20 total (Ollama free for dev, OpenAI only for final demo polish)
Key milestone	First LinkedIn post with live demo video — job search signal begins

Sprint 2 — LangGraph System + Interceptor

Goal: Working 3-agent LangGraph system with AgentGuard interceptor logging every tool call to SQLite in real time

- Build 3-agent LangGraph scenario: email reader → CRM lookup → response writer with realistic tool calls
- Implement BaseCallbackHandler interceptor — on_tool_start (pre-execution) and on_tool_end (post-execution) hooks
- SQLite event store: timestamp, agent_id, tool_name, input_hash, output_hash, latency_ms, risk_flags
- Loguru structured JSON logging — never print(), always structured events
- Fail-open design: if interceptor raises exception, tool call proceeds with WARNING log — agents never blocked
- Streamlit trace dashboard v1 — live activity feed, filter by agent / tool / time / risk level
- Unit tests: test_interceptor.py, test_event_store.py with mock_llm fixture
- Record 2-min Loom — 'I built visibility for AI agents' — post to LinkedIn

Sprint deliverable: Live Streamlit dashboard showing agent tool calls in real time — first shareable demo moment

Sprint 3 — Policy Engine + DLP Interceptor

Goal: YAML policy engine enforcing rules + PII exfiltration blocked live — the core demo moment

- Design YAML policy schema: rules with match criteria (tool, domain, data_pattern, direction) + action (allow/block/alert)
- Implement PolicyEngine class: load YAML at startup, evaluate rules per tool call pre-execution, return Decision
- Wire PolicyEngine into interceptor — block/allow/alert decision before tool executes, latency target <50ms
- Integrate Microsoft Presidio for PII detection — SSN, CC, PHI, email, phone — runs 100% locally on M1
- Build PII_EXFIL_001 demo scenario: agent tries to POST customer SSNs to external endpoint → blocked + alerted
- Unit tests: test_policy_engine.py (all rule types), test_dlp_scanner.py (synthetic PII fixtures)
- Integration tests: test_interceptor_policy.py, test_policy_dlp.py
- Dashboard: add red-highlighted block/alert events, policy violation timeline view

Sprint deliverable: PII exfiltration blocked live in demo — 'That's your DLP, but for AI agents'

Sprint 4 — Injection Detection + Audit Chain + OSS Launch

Goal: Complete 4-module MVP — GitHub OSS launch, 5-min demo recorded, job search signal begins

- Prompt injection detector: regex pattern matching + Ollama LLM classifier for edge cases
- Build INJECT_INDIRECT_001 scenario: malicious support ticket caught before agent processes instruction
- Tamper-evident audit chain: SHA-256 chaining — each entry hashes previous, verify_chain() CLI command
- PDF audit report export via fpdf2: violation summary, timeline, agent activity stats — SOC2-ready format
- Adversarial tests: test_fail_open.py (crash policy engine), test_audit_tamper.py (modify log → detection)
- Full GitHub README: architecture diagram, install guide, demo GIF, coverage badge, latency badge
- Record 5-min Loom covering all 4 modules — the full demo narrative script rehearsed 3+ times
- HN Show HN post, LinkedIn post, r/LangChain — first GitHub stars
- Blog post: 'Why your AI agents need a governance layer' published on Medium
- ATTACK_MATRIX.md: map all scenario tests to OWASP LLM Top 10 categories

Sprint deliverable: AgentGuard MVP live on GitHub — public, demo-ready, enterprise-grade foundations baked in

SECTION 7

Option A · Phase 1 — Governance Layer (Sprints 5–8)

Sprints	5, 6, 7, 8 — 4 weeks
Goal	Enterprise pilot-ready: FastAPI, Postgres, RBAC, /metrics, SIEM webhook
Job search	Active from Sprint 8 — 15 warm outreach messages sent
Milestone	First paid pilot or job interview at an AI security company

Sprint 5 — FastAPI Backend + Authentication

Goal: REST API wrapping the core library, JWT auth, RBAC — pilot-ready security model

- Build FastAPI backend with Uvicorn — REST API wrapping AgentGuard core library
- Endpoints: GET /events, GET /events/{id}, POST /policies, GET /audit/export, GET /health
- JWT authentication — short-lived tokens for dashboard, long-lived API keys for integrations
- RBAC: admin (edit policies), auditor (read-only logs), viewer (dashboard only)
- Wire Streamlit dashboard to call FastAPI instead of querying SQLite directly
- test_fail_open.py: verify agents keep running when the FastAPI layer is unreachable
- Update LinkedIn profile — pin demo video and GitHub stars badge

Sprint deliverable: FastAPI backend deployed locally, dashboard wired to API

Sprint 6 — Postgres + Data Layer Migration

Goal: Production event store with Alembic migrations and async Redis queue

- Migrate from SQLite to PostgreSQL 15 via Docker — EventStore abstraction class unchanged
- Alembic migration system — all schema changes as versioned files, never manual SQL
- Indexes on agent_id, timestamp, policy_result — query p99 <100ms for 1M events
- Redis async queue — events buffered in memory, DB write in background thread — latency unchanged
- Data retention policies — configurable purge schedule, GDPR right-to-erasure per session
- Seed script: 10,000 synthetic events for staging — confirms dashboard stays responsive at scale
- Integration test: test_full_pipeline.py at 10K events — performance validated

Sprint deliverable: Postgres running in Docker, migration system live, async logging verified

Sprint 7 — Docker + Staging + Prometheus Metrics

Goal: Enterprise deploy story: one docker-compose up and everything runs

- Docker image for AgentGuard core + API + dashboard — multi-stage build, minimal image size
- docker-compose.yml: single command spins up AgentGuard + Postgres + Redis + dashboard
- Prometheus-compatible /metrics endpoint: events/sec, block rate, p50/p95/p99 latency
- /health endpoint: JSON system status — database connection, queue depth, version
- Locust performance test suite — validate 100 events/sec sustained, 10 concurrent agents
- First performance benchmark published: PERF_BENCHMARKS.md with real measured p99 numbers
- weekly-perf.yml GitHub Actions job wired up — runs Locust every Monday

Sprint deliverable: Docker Compose staging environment live, performance numbers documented

Sprint 8 — SIEM Integration + Job Search Sprint

Goal: Security events flow into Splunk/Datadog + active outreach to 15 companies

- Webhook output in CEF format — compatible with Splunk, Datadog, Microsoft Sentinel
- Configurable alert channels: email, Slack webhook, PagerDuty integration
- All scenario tests passing: PII_EXFIL_001–005, INJECT_001–010, UNAUTH_API_001–005
- Full adversarial test suite green: crash recovery, tamper detection, all edge cases
- Evaluation suite first run — establish baseline.json for regression tracking
- Send 15 warm outreach messages: Palo Alto Networks, Cisco AI, HiddenLayer, Protect AI, Robust Intelligence, Anthropic, Scale AI, CalypsoAI, Cohere, Salesforce Agentforce
- Resume updated: AgentGuard as top entry with GitHub + demo link prominent
- Freelance AI security audit offer: $150–200/hr — target first engagement

Sprint deliverable: SIEM integration live + job search pipeline started with 15 outreach messages

SECTION 8

Option A · Phase 2 — Pilot Ready (Sprints 9–12)

Sprints	9, 10, 11, 12 — 4 weeks
Goal	30-day enterprise pilot: SSO, Kubernetes, compliance reports, HA mode
Target	First paying design partner OR active interview process at AI security company
PyPI	pip install agentguard — public package release via GitHub Actions

Sprint 9 — SSO + Enterprise Authentication

Goal: SAML/OIDC integration — enterprise IT uses their existing Okta or Azure AD

- SAML 2.0 / OIDC integration — works with Okta, Azure AD, Google Workspace out of box
- No separate user management — enterprise IT plugs in their existing identity provider
- JWT refresh token flow, configurable session expiry, secure token rotation
- Audit log entries include authenticated user identity — required for SOC2 CC6
- Test: enterprise login flow, permission boundary enforcement, token expiry handling

Sprint deliverable: SSO working — enterprise IT can deploy without managing separate user accounts

Sprint 10 — Compliance Report Templates

Goal: One-click PDF reports mapped to SOC2, HIPAA, ISO 27001 — what auditors need

- SOC2 CC6 template — AgentGuard events mapped to specific CC6 control requirements
- HIPAA §164.312 template — PHI access log, minimum necessary access evidence
- ISO 27001 A.12.4 template — operational logging and monitoring evidence
- One-click PDF export from dashboard — formatted for direct auditor submission
- AUDIT_001–003 scenario tests passing: 30-day audit export verified complete
- Blog post: 'How AgentGuard makes your AI agents SOC2-ready'

Sprint deliverable: Compliance reports generated and verified against real framework requirements

Sprint 11 — Kubernetes + Helm Chart

Goal: Production enterprise deploy — one helm install in any K8s cluster

- Helm chart — single command deploy to EKS, GKE, or AKS
- Resource limits, liveness/readiness probes, horizontal pod autoscaling config
- Stateless interceptor design confirmed — multiple instances behind load balancer
- PostgreSQL read replica — no single point of failure in data layer
- ConfigMap for policy files — DevOps manages security rules via K8s
- Tested: rolling deploy with zero downtime while agents are running

Sprint deliverable: Helm chart published, K8s deploy tested — enterprise production deployment story complete

Sprint 12 — HA Mode + PyPI Release

Goal: 10K events/min validated + pip install agentguard on PyPI

- HA mode: stateless interceptors + Redis shared session state, no SPOF
- Validated at 10K events/min — load test results published in README
- PyPI package release: pip install agentguard — automated via GitHub Actions release tag
- AI Engineering Summit talk submission: 'Building DLP for AI agents from a network security background'
- Evaluation suite: injection precision >0.90, DLP FP rate <2% — published as README badges
- Freelance AI security audit: target 1–2 engagements at $150–200/hr to bridge income

Sprint deliverable: AgentGuard on PyPI, HA mode production-validated, evaluation metrics published

SECTION 9

Option A · Phase 3 — Production Scale (Sprints 13–16)

Sprints	13, 14, 15, 16 — 4 weeks
Goal	Production v2: Next.js dashboard, policy marketplace, runtime sandbox beta
Milestone	Paying enterprise customer OR job offer at an AI security company
Next	Option C development begins immediately after Sprint 16 completes

Sprint 13 — Next.js Enterprise Dashboard

Goal: Replace Streamlit with production-grade React dashboard — enterprise UI standard

- Migrate Streamlit dashboard to Next.js 14 + Tailwind CSS + shadcn/ui
- Real-time WebSocket event feed — live updates without page refresh
- Recharts/Tremor analytics: event timelines, violation heat map, latency charts
- Role-based dashboard views — SOC analyst vs compliance officer vs admin
- Mobile-responsive — CISO can monitor agent activity from phone
- Deploy to Vercel under dashboard.stridemap.ai or as subdirectory

Sprint deliverable: Production Next.js dashboard live — professional enterprise UI

Sprint 14 — Policy Marketplace

Goal: Pre-built policy templates + community rule sharing — the moat builder

- Policy template library: finance, healthcare, legal, IT pre-built rule sets
- Community submission flow — enterprises share anonymised policy rule sets
- Policy version control — track changes to rules over time with diff view
- Policy testing sandbox — test a rule against synthetic data before deploying live
- 'Policy as Code' documentation — treat security rules like infrastructure code (gitops-friendly)
- First 3 contributed policies from early access users if available

Sprint deliverable: Policy marketplace live — pre-built rules for finance, healthcare, legal

Sprint 15 — Runtime Sandbox Beta

Goal: First step into Option C — OS-level isolation as an AgentGuard plugin

- Agent runtime isolation — sandbox individual tool executions at OS level (research phase)
- File system access control — agents restricted to approved paths only
- Network egress filtering at kernel level — enforce independently of policy YAML
- Process spawning prevention — agents cannot launch subprocesses
- Document the architecture in docs/ — this becomes the Option C foundation spec
- This is the technical bridge: governance layer evolves into full runtime security

Sprint deliverable: Runtime sandbox beta — Option C architecture validated, first OS-level constraints working

Sprint 16 — Option A Completion + Option C Transition

Goal: All quality gates met, Option C sprint plan ready, retrospective published

- Full regression test suite green across all 5 layers — no P0 or P1 bugs open
- ATTACK_MATRIX.md: >80% OWASP LLM Top 10 coverage confirmed and published
- Performance SLAs all met: p99 <50ms, >100 events/sec — documented with real numbers
- Option C detailed architecture design complete — reuses interceptor + policy engine + audit chain
- Option C Sprint 17 kickoff plan written and ready
- Public retrospective blog post: 'What I learned building an AI agent governance layer in 16 sprints'
- GitHub stars milestone check — 500 stars target for credibility in Option C phase

Sprint deliverable: Option A v2.0 complete — production-ready, all gates passed, Option C begins

SECTION 10

Option C — Agent Runtime Security (Sprints 17–24)

Sprints	17–24 — 8 weeks, begins after Option A Phase 3 completes
Product	OS-level sandboxing, tool call isolation, network egress control at kernel level
Foundation reused	AgentGuard interceptor, policy engine, audit chain, RBAC, SSO — all inherited
Market timing	Enterprise buyers ready for runtime security after experiencing governance layer value

Option C is not a separate product — it is the natural technical evolution of Option A. The interceptor architecture that captures tool calls for policy evaluation becomes the hook point for OS-level sandbox enforcement. The policy engine that evaluates YAML rules becomes the source of truth for what sandboxes allow. The audit chain that logs governance events is extended to capture runtime security events. Everything compounds.

The Option A → Option C technical bridge: Sprint 15 of Option A includes a runtime sandbox beta as an AgentGuard plugin. This gives real-world validation of the approach and means Option C begins with a working prototype, not from scratch.

What Option C adds

Capability	What it does
Secure agent sandbox	Isolated execution environment per agent tool call — failures contained, no lateral movement
OS-level tool call control	seccomp/eBPF filters restrict what system calls an agent tool execution can make
File system access control	Agents restricted to declared paths — any other access attempt blocked at kernel level
Network egress sandboxing	Outbound connections validated at OS level — independent of the policy YAML layer
Process spawn prevention	Agent tool calls cannot fork child processes or execute shell commands
Reasoning + action audit	Chain-of-thought captured alongside tool calls — not just what agent did but why
Tool call allowlist enforcement	Only declared tools can be invoked — any undeclared tool call blocked at runtime
Anomaly detection	Baseline normal behaviour per agent, alert on deviations — ML-based in v2
Tamper-proof runtime log	Extends AgentGuard audit chain — every sandbox event chained cryptographically
SIEM + SOC integration	Runtime security events alongside governance events in Splunk/Datadog — unified view

Option C sprint plan

Sprints 17–18	**Foundation**	seccomp/ptrace research, sandbox architecture design, file system isolation prototype, tool allowlist enforcement at OS level
Sprints 19–20	**Network + Process**	Network egress sandboxing, process spawn prevention, performance impact measurement — target <20ms additional overhead
Sprints 21–22	**Audit + Detection**	Reasoning capture alongside tool calls, anomaly detection baseline, extend tamper-proof audit chain to runtime events
Sprints 23–24	**Launch + GTM**	Integration with Option A dashboard (new Runtime Security tab), pilot with 2 enterprise design partners, pricing model, stridemap.ai platform page updated

Technical approach: Start with seccomp (Linux syscall filtering) and ptrace-based monitoring — well-understood, stable, zero additional dependencies. Research eBPF for more granular control in v2. gVisor as an alternative sandbox runtime is evaluated in Sprint 17 but likely deferred to v3 due to performance overhead.

Performance target for Option C: Combined overhead (Option A governance + Option C sandbox) under 80ms p99 per tool call. This is the enterprise SLA for the full runtime security stack.

SECTION 11

Go-to-Market & Job Search Strategy

Target roles — Bay Area

AI Security Engineer / Founding Eng.	HiddenLayer, Protect AI, CalypsoAI, Robust Intelligence	**Strong fit now**	$180–260K + equity
Staff AI Platform / Infra Engineer	Anthropic, Cohere, Together AI, Mistral, Groq	**Strong fit now**	$220–320K
AI Safety / Red Team Engineer	Anthropic, Google DeepMind, Meta AI, Scale AI	**Fit from Sprint 8**	$220–340K
Enterprise AI Platform Engineer	Salesforce Agentforce, Cisco AI, Palo Alto Cortex	**Warm intro path**	$200–290K
Agent Runtime Security Engineer	Anthropic, OpenAI, NVIDIA, Cloudflare Workers AI	**Fit from Sprint 20**	$240–360K

Build in public — content calendar

- Sprint 1 Fri: 'stridemap.ai is live — building AI agent governance'
- Sprint 2 Fri: 2-min Loom — 'I built real-time visibility for LangGraph agents' + dashboard GIF
- Sprint 3 Fri: 'I built DLP for AI agents — here's how it works' + live PII block demo clip
- Sprint 4 Fri: 'AgentGuard is live on GitHub' + full 5-min demo Loom + HN Show HN
- Sprint 8: Blog post — 'Why your AI agents need a governance layer' on Medium + dev.to
- Sprint 12: AI Engineering Summit talk submission — 'Building DLP for AI agents from a NGFW background'
- Sprint 16: Retrospective blog — '16 sprints building AI agent governance as a solopreneur'
- Sprint 20: DEF CON AI Village talk submission — 'Runtime Security for AI Agents: From seccomp to eBPF'

Outreach strategy

- Target: ex-Cisco and ex-Palo Alto engineers now at AI startups — warmest possible context, you speak their language
- Message format: 3 lines max — 'Built this, here's the GitHub + 5-min demo. Would love your feedback.' Ask for feedback first, not a job.
- Sprint 8: Send 15 warm outreach messages. 8 to AI security startups, 7 to big tech AI teams
- Follow and engage meaningfully on LangChain, LangGraph, HiddenLayer, Protect AI LinkedIn and Twitter/X
- Freelance bridge: AI security audit at $150–200/hr — one $5K engagement funds 6 months of running costs
- OSS community: respond to GitHub issues within 24 hours, merge PRs visibly — public contributor reputation builds fast

SECTION 12

Daily Routine & Development Workflow

Full-time weekdays only — weekends protected completely. This is not negotiable. Solo sprint burnout at week 3 is the single most common failure mode for projects like this. The structure below is designed to produce consistent daily output without degradation.

Daily schedule — Mon through Fri

Time	Activity	Purpose
9:00–11:00am	Vibe code with Cursor	Build the day's module — let AI scaffold, you direct architecture and review output
11:00–12:00pm	Read and understand	Read the generated code line by line. Ask Cursor/Claude to explain each block you don't fully grasp
12:00–1:00pm	Lunch break — no screens	Brain rest is non-negotiable. Skipping this degrades afternoon output quality
1:00–2:30pm	Research and learn	Read one doc, article, or paper on today's topic: LangGraph internals, OWASP LLM Top 10, seccomp, NGFW DLP
2:30–4:00pm	Test, break, and debug	Write one test case. Break something deliberately. Fix one bug manually (not vibe-coded). Builds real understanding.
4:00–5:00pm	Document and share	Write a README section, update architecture notes, draft a blog paragraph, or post a LinkedIn update

Vibe coding rules

- Vibe code to build fast — use Cursor with full codebase context (@file references) for maximum speed
- After each coding session: spend 30 minutes understanding what was generated and why
- Two modules require deep understanding (not just running): the Interceptor and the Policy Engine — these are what interviewers drill into
- Everything else can be scaffolded and explained at architecture level: 'I used Presidio for PII detection because it runs locally, here's how it integrates'
- Never commit code you can't explain the purpose of — 'make test' must pass before any push
- The interview answer to 'did you write this yourself?': 'I used AI-assisted coding to scaffold quickly, then studied and tested every module. I own the system design and security logic.' This is honest and respected in 2025.

M1 Pro workflow note: Run Ollama as a background service on startup — `ollama serve`. All LLM calls during development hit localhost:11434 automatically via the LLMClient abstraction. Zero API cost, zero network latency, full privacy. Only switch providers by changing one env var when preparing the final demo.

SECTION 13

Cost & Resource Plan

Phase	Monthly cost	Notes
MVP Sprints 1–4	~$20 total	Ollama (free), Gemini free tier (dev), ~$20 OpenAI for demo polish only
Option A Phase 1 (5–8)	~$15/month	OpenAI API ($10), domain ($1.50/mo), Streamlit Cloud (free)
Option A Phase 2 (9–12)	~$55/month	Above + LinkedIn Premium ($40) during active job search sprint only
Option A Phase 3 (13–16)	~$25/month	API costs, Render/Railway hosting (~$7), LinkedIn Premium cancelled
Option C (17–24)	~$35/month	Increased API usage for runtime testing, Render/Railway hosting
Total 6 months	~$200 total	The real investment is ~250 hours of your time, not cash

Development environment

Tool	Details
Hardware	MacBook M1 Pro 16GB — runs all tools natively including Ollama local LLM
IDE	Cursor — purpose-built for vibe coding, free tier, full codebase context
Python	3.11 via pyenv, .venv per project, ARM64 native on M1
Local LLM	Ollama llama3.2 — 30+ tokens/sec, free, private, zero API cost during dev
Staging	Docker Desktop (free), Postgres via Docker, ngrok for ad-hoc demo sharing
Hosting	Vercel (free tier) for web/, Streamlit Cloud (free) for dashboard

The total cash cost of this entire roadmap — MVP through Option C — is approximately $200. The meaningful investment is time: roughly 250 weekday hours across 6 months. The question is not 'can I afford to build this?' — it is 'will 250 focused hours on this get me to a $200K+ role faster than anything else I could do?' Given the directness of the skills match, the answer is strongly yes.

SECTION 14

Risk Register

Large vendors ship similar OSS tool	**Medium**	**Medium**	OSS head-start, NGFW practitioner credential is hard to fake, LangGraph-native niche, policy marketplace community moat
LangGraph API changes break interceptor	**Medium**	**High**	Pin LangGraph version in requirements.txt, integration tests catch breaks on every push, abstract interceptor interface
Burnout from solo full-time build	**Medium**	**High**	Strict weekends-off policy, 9am–5pm structure only, progress LinkedIn posts for public accountability
Live demo fails in an interview	**Medium**	**High**	make demo Makefile target runs all scenarios, offline Ollama fallback for LLM, pre-recorded Loom backup always ready
No enterprise interest at MVP	**Low**	**Medium**	Warm intros via Cisco/Palo Alto network, freelance audits build credibility, HN launch drives organic inbound
Presidio PII accuracy insufficient	**Low**	**High**	Hybrid approach: Presidio + LLM classifier for edge cases, enterprise-tunable patterns, published precision/recall metrics
Option C too technically complex solo	**Medium**	**Medium**	Sprint 15 Option A includes runtime sandbox prototype — Option C begins with working code not from scratch
Job offer arrives mid-Option A	**Low**	**Low**	Take the job — Option A becomes SaaS side income. Option C slides to post-employment side project.

SECTION 15

Success Metrics & Milestones

Product milestones

Milestone	Definition of done
End of Sprint 1	stridemap.ai live with HTTPS, GitHub repo public, CI green, LinkedIn updated
End of Sprint 4 MVP	AgentGuard on GitHub, 5-min demo live, HN post published, first GitHub stars
End of Sprint 8	15 outreach messages sent, 3+ job conversations started, Docker deploy live
End of Sprint 12	pip install agentguard on PyPI, first freelance audit, conference talk submitted
End of Sprint 16	Paying design partner OR job offer at AI security company — Option C begins
End of Sprint 20	OS-level sandbox working with <20ms additional overhead, DEF CON talk submitted
End of Sprint 24	Option C beta with 2 enterprise pilots OR full-time role secured + side product

Technical quality gates

Metric	Target	Gate type
Test coverage on core modules	> 80%	Blocks merge to main
Injection detector precision	> 0.90	Blocks release
Injection detector recall	> 0.85	Blocks release
DLP false positive rate	< 2%	Blocks release
DLP false negative rate	< 1%	Blocks release
Latency p99 (Option A)	< 50ms	Documented SLA
Latency p99 (Option A + C combined)	< 80ms	Documented SLA
Throughput sustained	> 100/sec	Advisory metric
OWASP LLM01–05 coverage	> 80%	Advisory metric
P0 bugs open at release	0	Hard gate

Career success metrics

Metric	Target

GitHub stars (agentguard)	50 by Sprint 4 → 200 by Sprint 8 → 500 by Sprint 12
Job interviews from outreach	> 30% conversion rate from warm outreach with demo link
Target compensation	$200K+ base at AI security startup or big tech AI team
Freelance audit rate	$150–200/hr, minimum 1 engagement by Sprint 12
LinkedIn profile views	Measurable increase after each demo post — signal of visibility

stridemap.ai

Charting a secure path through AI infrastructure

hello@stridemap.ai | github.com/stridemap/agentguard